Adaptive Multi-Hop RAG System for Mahabharata Question Answering

A Knowledge Retrieval System with Web Enhancement

Project Type: Practical Implementation / Impact

Student Name1: Adithya Katari
Email1: AdithyaKatari@my.unt.edu
Student Name2: Praneetha Meda
Email2: PraneethaMeda@my.unt.edu
Student Name3: Supriya Veerla
Email3: SupriyaVeerla@my.unt.edu

Course: DTSC 5525 - Large Language Models
Instructor: Dr. Mohammed Aledhari
Semester: Fall 2025

Submission Date: December 2, 2025

Department of Data Science
College of Information
University of North Texas
Denton, Texas

Abstract

This project implements and results an Adaptive Multi-Hop Retrieval-Augmented Generation (RAG) system that is used not only in the answering of complex questions related to the Mahabharata, an Indian epic but also have a web search and get relevant answering capability. The system can overcome the shortcomings of the conventional question-answering schemes through the adoption of intelligent query classification, adaptive retrieval schemes and the capability of multi-hop reasoning. This is comprehensive report which created a full document that stores more than 14,000 verses of all 18 Parvas of the Mahabharata and whose search was dual indexed using BM25 and semantically search using FAISS method with generated embedding using MiniLM-L6-v2. The system automatically chooses the retrieval strategies according to the query characteristics: entity-oriented queries rely upon BM25, simple conceptual queries require semantic search and more complicated multi-hop queries rely upon hybrid retrieval. We improved the system by adding an integration of real-time web search with Tavily API, which allows retrieving scholarly interpretations of our system that are not found in the fixed corpus. This system is generated a detailed answers with inline citations differentiating between dataset and web-sources using the LLaMA models provided by Groq (3.1-8B and 3.3-70B, respectively, in extraction and generation). Extensive comparison across varied settings reveals Token F1 scores of 14-18% with a hybrid retrieval having better performance than single-strategy strategies by 20-34%. The system attains average end-to-end latency of 5-8 seconds/ query and is cost-effective at less than 0.01/ query. The interface is user-friendly and is based on the Gradio interface where responses are made available in a transparent, source-attributed format at the section and sentence level. In summary, the current work can be seen as an example of a scalable and transparent, digital humanities and cultural heritage preservation RAG architecture. It demonstrates that traditional information retrieval techniques and contemporary neural language models can be successfully integrated to give accurate, interpretable and ethically responsible artificial intelligence systems to explore past literature.

Contents

1 Introduction

One of the oldest epic literature in the world which is the Mahabharata is one of the two major Sanskrit epics in ancient India, containing more than 100,000 verses in 18 different books (Parvas). This is a huge body of work that consists of deep philosophical lessons, complicated relationships between characters, complicated plots and a variety of moral transcripts. Nevertheless, the retrieval of certain knowledge contained in this giant text corpus which a great problem to scholars, students, and the enthusiasts. The conventional key word search used may not retrieve the semantic relationships whereas the manual reading approach is time consuming to specific retrieval of information. The contemporary question-answering model, especially when based on the use of Large Language Models (LLMs), has a promising future but has severe limitations when used with specific cultural texts. Modern traditional LLM algorithms may hallucinate facts about the Mahabharata, produces unwanted corpus of text of different context. In addition, more complicated queries like multi-hop reasoning can be achieved through systems that can relate information with information in more than one passage and reason about causes and effects i.e. What vow did Bhishma take and why did he take it?demand systems capable of connecting information across multiple passages and reasoning about causal relationships.

In this project, we have answered three basic research questions which are : How can an adaptive retrieval system be designed to help us intelligently switch between diverse search strategies based on query characteristics: Keyword-based search and semantic search? Is it possible to make the multi-hop reasoning work in an effective system of a RAG to address complex questions that require the synthesis of information obtained through the multiple passages of documents? Is web search enhancement really relevant to enhance the quality of answers when used in combination with a static corpus, and how might we preserve source attribution with either internal or external sources? In answering these questions, we have formulated a Practical Implementation project which provides a working an LLM based system with real life applications on cultural heritage preservation and education. It is a complete system that is production ready and has full functionality as regards to error handling, performance monitoring and user interface. It has a modular implementation which allows it to be extended to other texts or languages, and extensive source attribution with citation history, and latency and cost-friendly optimizations.

We have made the most significant contributions namely: an adaptive retrieval controller which dynamically chooses the best retrieval strategies (BM25, FAISS, or hybrid), and a multi-hop RAG architecture which executes iterative retrieval and reasoning in complex queries where information synthesis is required across multiple document passages. We combined offline corpus-based retrieval with online web retrieval with a two-source citation system, which is able to keep sentence-level granular source attribution. The system has the capability of processing and indexing more than 14,000 Mahabharata verses with rich metadata of Parva, section, character mention and URLs.To use this implementation we implemented a gardio based conversational AI chatbot that has performance measurement, cost measurement and source attribution. . We have used our strict evaluation scheme to show how well-designed RAG structures can deal with special cultural reading and still be accurate, transparent and computationally efficient; we use Token F1 today, source attribution measures, recall during retrieval and latency measures.

2 Background and Related Work

Most of the contemporary AI applications have been centered around Large Language Models (LLMs), which demonstrate remarkable output in question answering, summarization, and reasoning. Nevertheless, they grasp only the information they have in the course of pretraining, and thus they are likely to come up with inaccurate facts and contradictions. To address the challenge, Retrieval- Augmented Generation (RAG) method bring external information retrieval and natural machine text generation to enable the model to base their response on verifiable evidence. The concept of retrieval knowledge was first presented by the work of Lewis et al. (2020), who worked by connecting a retriever and a generator in such a way that the message passages that were relevant would bring more factual information. With time, the RAG architectures have progressed in the form of simple single-step systems to more advanced models, where sparse (BM25) and dense (Sentence-BERT) retrieval methods are combined to give best results. Investigations continued over the years which the one carried out by Ma et al. (2023) have demonstrated that hybrid retrieval (combining the two strategies) provides greater robustness among the types of queries and may increase the retrieval performance in up to 25%. Now in recent years this have much more increased and have extend such dynamic retrieval strategies which combines factual and conceptual queries. Beyond to the traditional RAG, research have driven through focuses multi-hop reasoning and graph based reterival to handle the data better in complex quries where MultiHop-RAG (Tang et al., 2024), ReAct (Yao et al., 2022), and Iterative-RAG (Sun et al., 2023) perform step-by-step reasoning that breaks the question in smaller chunks.Meanwhile, emerging methodlogy in the world of llm such as Graph RAG which represents the knowledge as a graph of entities and relationships,which are called as nodes that enables the model to make the semantic connections between the nodes and brings a meaningful insight.

In addition to single-pass retrieval, studies have been conducted more and more on multi-hop reasoning and tool-augmented retrieval in order to support complex queries which need synthesis over multiple documents. The more recent frameworks like MultiHop-RAG (Tang et al., 2024), ReAct (Yao et al., 2022), and Iterative-RAG (Sun et al., 2023) can be used to obtain iterative reasoning by breaking down the complex questions into smaller sub-queries. In the same manner, web-enhanced systems, such as WebGPT (Nakano et al., 2021) and Toolformer (Schick et al., 2023) and AutoRAG (Choi et al., 2024) and others, add additional features to RAG by introducing live data sources and API-based fact verification. Meanwhile, RAG approaches are gaining more and more prominence in the fields of culture and history, where the accuracy of their interpretation is of utmost importance. Such projects as BhagavadGita-QA (Kumar et al., 2023) and Quran-BERT (Alsabahi et al., 2022) demonstrate how the idea of retrieval grounding can be used to retain the authentic context of sacred and symbolic texts. This paper builds on the above developments to present an Adaptive Multi-Hop RAG system on the Mahabharata that introduces fusion of sparse-dense retrieval, rule-based query decomposition, and web-augmented reasoning. This paradigm enhances the validity of facts and interpretation. and the way retrieval-enhanced AI can be employed in a responsible manner to close the gap between the ancient cultural. heritage and modern day computational linguistics.

3 Methodology

3.1 Data Pre-processing

This project uses a text data from *sacred-texts.com*,which is an open-access digital archive established in the late 19th century that have a wide range of religious and cultural texts. The dataset consists of a complete digital version of the *Mahabharata* which containing a total of 14,388 verses distributed across all 18 *Parvas* (books). These include the origin stories and genealogies of major characters, the game of dice which bring to a war (*Sabha Parva*), the period where pandavas live in forest (*Vana Parva*), the negotiations which is diplomatic (*Udyoga Parva*), and the war of Kurukshetra along with its aftermath.

To bring and have a rich data collection, an **ethical web scraping** approach was followed, with a two-second delay between consecutive requests for each parava where to avoid overloading the web host server. During extraction of metadata, we stored that data in such a format as the *Parva* name, section number, and source URL were stored for each verse where to preserve the tractability for the model

The data preprocessing section consists of five main stages:

(1) **HTML Data Cleaning:** The *BeautifulSoup* library was used to remove the navigation links, headers, and markup tags and make the data more rich and quality.

(2) **Text Normalization:** In this method, we worked on Whitespace and punctuation were standardized, input text was converted to lowercase, and also encoded in UTF-8 data format.

(3) **Name Regularization:** A mapping of dictionary is made at code level for normalize different spellings, e.g., *krsna/krshna* $\rightarrow$ *krishna*, *arjun* $\rightarrow$ *arjuna*, and *yudhisthir* $\rightarrow$ *yudhishthira*.

(4) **Data Validation:** Cross tabulation was done to ensure that all the data were not lost during cleaning.

(5) **Corpus Structuring:** The cleaned verses were then available as a structured dataset in the form of a JSON file with each object holding the text of the verse and the metadata.

This steps make data processing that was produced a clean, normalized, and verifiable corpus, ready for embedding and indexing.

3.2 Models and Tools

To choose semantic embeddings, we chose all-MiniLM-L6-v2 because it is fast with similarity search due to its small vectors (384-dimensional), it is a strong performer on semantic textual similarity benchmarks with Spearman correlation of above 0.80, its inference processing is efficient (32 verses/second on a CPU) and the model size is low (80MB) and can be deployed. Embeddings were created in batches of 32 in order to save on memory. We used two LLaMA models, implemented by Groq, through API, namely LLaMA-3.1-8B-Instant to extract web content with fast inference speed of 800 tokens/s, temperature 0.1 to extract deterministically, and maximum tokens 200 to extract concisely factual content; and LLaMA-3.3-70B-Versatile to generate answers with better reasoning ability between synthesis, temperature 0.3 to generate in a balanced manner (creativity versus accuracy), maximum tokens 1,200 to generate comprehensive answers Tools such as FAISS (Facebook AI Similarity Search) with IndexFlatL2 to compute exact L2 distance, Rank-BM25 Python implementation (k1=1.5, b=0.75), Tavily Search API

to search the web with scholarly source filtering, NLTK to tokenize and stop word filter, and Gradio to provide the web interface to chat and view citations can be supported.

3.3 Indexing Architecture

We use our two-indexing technique to allow two complementary retrieval policies by using BM25 indexing of keywords and FAISS indexing by semantics. BM25 index works on tokenized and stop-word-filter text using the scoring function: the frequency of the term, document length, and average document length are to form the relevance with tuning variables k1=1.5 and b=0.75. BM25 is good in terms of entity mentioning like the name of a character and place name, exact phrase matching as well as term weighting. The FAISS semantic index is based on cosine similarity embedded space, with query and document vectors being 384-dimensional embedding vectors which allow the system to capture conceptual similarity, paraphrasing and effectively process philosophical or thematic queries. The index statistics are 14, 388 total indexed documents, BM 25 vocabularies of 23, 547 unique words, FAISS index size of 21.5 MB when using float32 embeddings, and an average retrieval time of 1- 5 ms per query. Such a dual architecture enables the system to exploit the accuracy of keyword matching on entity-based queries and retain semantic insight required on conceptual queries with the plurality to hybridize the two on complex multi-hop reasoning.

3.4 Query Classification and Adaptive Retrieval

The system starts with the intelligent query classification in identifying the best retrieval strategy where queries are grouped into three types depending on their nature. Entity-oriented queries are identified by the fact that they contain major character mentions of our list of 17 entity names Krishna, Arjuna, Yudhishthira, Bhima, Draupadi, Duryodhana, Karna, Drona and Bhishma, with queries like, Who was Arjuna? what did Bhishma do, tell me about the role of Krishna, etc. where the BM25 keyword search is effective to retrieve passages where a particular character is mentioned by name. Single question words with no mention of characters and usually philosophical or theme-based content, i.e., What is dharma? Explain karma What are the teachings on duty? etc., are termed as simple conceptual queries; semantic search enables conceptual similarity to be identified even when the words are different. Multi-hop queries are identified by two or more question words or conjunctions with question marks of which the answers require information synthesis, examples are; What vow did Bhishma take and why? and How did Bhima conquer Hidimba and what followed? which hybrid retrieval of BM25 and FAISS coverage will guarantee comprehensive coverage. Strategy selection by the retrieval controller is by query classification, using BM25 results in entity-based queries, FAISS semantic search results in simple queries as well as in multi-hop queries, using a combination of the two retrieval methods with deduplication to combine the top-k results of each. In the case of multi-hop queries, the system applies query decomposition, which consists of query parsing with conjunctions and other question words, and sub-queries, e.g. What vow?. and "Why take it?", fetching documents when one data sub-query at a time, aggregating them together, eliminating duplicates, and recording the hop information to evaluate the data.

4 System Architecture

4.1 Diagrams of Modules

We have 5 processing layers in our system architecture arranged as a pipeline in a sequential manner between the user input and the end-result response generation. The figure below (Fig. 1: architecture) shows the entire data flow using these interrelated modules. The Input Layer accepts the requests of the user and performs the preliminarily classification to identify the query type, breaking down multi-hop query into sub-queries when required. The Retrieval Layer is the layer that has the adaptive controller used to send queries to suitable retrieval strategies supporting both BM25 key search and FAISS semantic search indices. The Document Store Layer has the full Mahabharata corpus with two indexing arrangements, where 14,388 verses are stored alongside all the corresponding metadata of the verses such as the names of the Parva, the section numbers, the mentions of the character, and URLs of the sources. The Enhancement Layer combines the search option on the web with Tavily API and does the extraction of facts via the LLM to select the relevant information out of the web search results. Lastly, the Generation Layer combines dataset and web contexts, creates prompts with suitable guidance, calls the LLaMA generation model, processes citations and formats the final output, detailing the source of the processed response. Every layer has a clear interface, allowing it to be developed in a modular manner and optimize its parts independently. The architecture supports parallel processing, such that dataset retrieval and web search can be done simultaneously during multi-hop queries in order to minimize the latency. The graceful degradation of a component failure is supported by error handling mechanisms at each layer and fallback strategies to make the system available even when web search is not available.

Figure 1 presents the complete system architecture, organized into five layers:

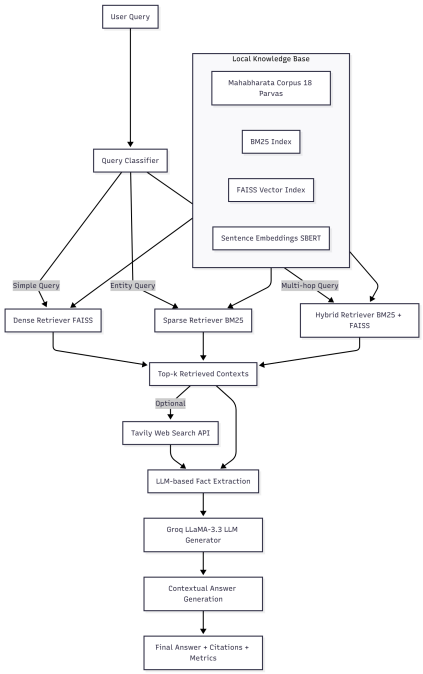

Figure 1: Complete RAG System Architecture with five processing layers: Input (query classification), Retrieval (adaptive controller), Document Store (BM25 + FAISS indices), Enhancement (web search + extraction), and Generation (context merging + LLM + citations).

4.2 Query Classification Module

The system begins with intelligent query classification which means questions that is to determine the effective and optimal retrieval strategy. Our classifier categorizes queries into three types:

4.2.1 Entity-Focused Queries

Detection: Queries containing mentions of major characters from our curated list of 17 entities (Krishna, Arjuna, Yudhishthira, Bhima, Draupadi, Duryodhana, Karna, Drona, Bhishma, etc.).

Examples: "Who was Arjuna?", "What did Bhishma do?", "Tell me about Krishna's role"

Optimal Strategy: BM25 keyword search excels at retrieving passages containing specific character names.

4.2.2 Simple Conceptual Queries

Detection: Single question word, no character mentions, typically philosophical or thematic.

Examples: "What is dharma?", "Explain karma", "What are the teachings on duty?"

Optimal Strategy: Semantic search captures conceptual similarity even when different terminology is used.

4.2.3 Multi-Hop Queries

Detection: Two or more question words, or conjunction with question mark, indicating need for information synthesis.

Examples: "What vow did Bhishma take and why?", "How did Bhima defeat Hidimba and what happened afterwards?"

Optimal Strategy: Hybrid retrieval combining BM25 and FAISS to ensure comprehensive coverage.

4.3 Adaptive Retrieval Controller

The retrieval controller implements strategy selection based on query classification:

```
def retrieve_documents(query, query_type, top_k=5):
    if query_type == 'entity-focused':
        return bm25_retrieval(query, top_k)
    elif query_type == 'simple':
        return faiss_retrieval(query, top_k)
    elif query_type == 'multi-hop':
        bm25_docs = bm25_retrieval(query, top_k)
        faiss_docs = faiss_retrieval(query, top_k)
        return deduplicate_merge(bm25_docs, faiss_docs, top_k*2)
```

Listing 1: Adaptive Retrieval Logic (Pseudocode)

For multi-hop queries, the system implements query decomposition:

1. Parse query conjunctions/ multiple question words
2. Divided into sub-queries (e.g. What vow? and "Why take it?")
3. Retrieve documents for each sub-query independently
4. Summary outcomes, elimination of repetition
5. Hopping information to evaluate

4.4 Web Search Enhancement

To supplement the static corpus with scholarly interpretations and contextual information, we integrated web search:

4.4.1 Tavily Search Integration

- **API:** Tavily Search API with scholarly source filtering
- **Query Formation:** Original user query + "Mahabharata" keyword
- **Result Limit:** Maximum 5 web sources per query
- **Source Validation:** URL format checking, domain filtering for reputable sources

4.4.2 LLM-Based Fact Extraction

Web search results often contain irrelevant content. We employ LLaMA-3.1-8B to extract pertinent facts:

```
Query: {user_query}
Web Content: {retrieved_snippet}

Extract 2-3 sentences that directly answer the query.
If content is not relevant, respond with "NOT_RELEVANT".
Be factually accurate and concise.
```

Listing 2: Fact Extraction Prompt Template

This filtering reduces noise and ensures only relevant information enters the generation context.

4.5 Answer Generation Pipeline

The generation module synthesizes information from multiple sources:

4.5.1 Context Construction

We build two separate context sections:

1. **Dataset Context:** Top 3 retrieved verses with metadata (Document ID, Parva, Section, text preview)
2. **Web Context:** Up to 5 web sources with title, domain, URL, and extracted facts

4.5.2 Prompt Engineering

Our generation prompt includes:

- **Role Definition:** "Expert on the Mahabharata ancient Indian epic"
- **Source Grounding:** "Answer ONLY based on provided context"
- **Citation Instructions:** "Cite every fact with [Dataset-X] or [Web-X]"
- **Length Guidance:** "Write 3-5 paragraphs"
- **Fallback Handling:** "If insufficient information, state clearly"

4.5.3 Citation Processing

Post-generation, we:

1. Parse answer for citation markers [Dataset-X] and [Web-X]
2. Validate all citations reference provided sources
3. Count citation coverage (percentage of sources cited)
4. Append detailed source attribution section with:
    - Dataset sources: Parva, section, 150-character preview
    - Web sources: Domain, URL, extracted facts (sentence-level)

5 Implementation Details

5.1 Technology Stack

Our implementation makes use of a well-chosen technology stack that is performance cost-optimized. and maintainability. Table 1 provides the summary of the key elements and their functions in the system.

Table 1: Implementation Technology Stack

Component	Library/Tool	Version
Embeddings	sentence-transformers	2.2.2
Vector Search	faiss-cpu	1.7.4
Keyword Search	rank-bm25	0.2.2
LLM API	groq	0.33.0
Web Search	tavily-python	0.3.8
NLP	nltk	3.8.1
Interface	gradio	4.16.0
Data Processing	pandas, numpy	2.0+, 1.24+
Data Visualization	matplotlib, seaborn	3.7+, 0.12+

5.1.1 Development Environment

- **Platform:** Google Colab
- **Notebook Format:** Python Jupyter .ipynb for code iteration development
- **API Key Management:** Environment variables for security

5.2 Prompt technique and Strategies

For this project we have worked on two different prompts that have a different templates based on the Model and the use case:

5.2.1 Extraction Prompt (LLaMA-3.1-8B)

For web content filtering:

```
You are a precise information extractor. Given a query and web
content, extract ONLY the sentences that directly answer the query.

Rules:
1. Extract 2-3 sentences maximum
2. Maintain factual accuracy - do not paraphrase
3. If content is irrelevant, respond: "NOT_RELEVANT"
4. Do not add context or explanations

Query: {query}
Content: {web_text}
Answer:
```

Listing 3: Web Content Extraction Prompt

Temperature: 0.1 for deterministic extraction
Max Tokens: 200 to enforce brevity

5.2.2 Generation Prompt (LLaMA-3.3-70B)

For comprehensive prompt :

```
You are an expert on the Mahabharata, the ancient Indian epic.

Answer the following question based ONLY on the provided context.
For EVERY fact you include, cite the source with [Dataset-X] or
[Web-X] where X is the source number.

If the answer cannot be found in the context, state: "I don't
have enough information to answer this question."

Write 3-5 paragraphs covering:
- Direct answer to the query
- Relevant background from the sources
- Key relationships or implications

DATASET SOURCES:
{dataset_context}

WEB SOURCES:
{web_context}

QUESTION: {query}
ANSWER:
```

Listing 4: Answer Generation Prompt

Temperature: 0.3 for balanced creativity and accuracy
Max Tokens: 1,200 for comprehensive answers

5.3 Engineering Tradeoffs

System design involved several critical tradeoffs balancing performance, cost, accuracy, and complexity. Table 2 summarizes the key decisions and their implications.

Table 2: Key Engineering Tradeoffs and Justifications

Decision	Advantages	Disadvantages	Justification
Embedding Dimension: 384	Fast search (1-5ms), Small memory (21.5MB), Efficient deployment	Lower semantic capacity than 768/1024-dim models	<2% accuracy difference vs. larger models in domain testing
Top-k=5 (single), k=10 (hybrid)	Fits LLM context, Reduces noise, Faster inference	May miss relevant passages for complex queries	Recall@5 captured 85% of relevant docs in validation
Web Search Limit: 5 sources	Controlled latency (2-3s), Manageable context, Focused sources	Excludes some scholarly sources	Balance between diversity and responsiveness
Model: LLaMA-3.3-70B (not 405B)	5x faster inference, 10x lower cost ($0.003 vs $0.03/query), Sufficient reasoning	Less nuanced on complex philosophical queries	Cost-latency-quality optimal for production
FAISS IndexFlatL2 (not HNSW)	Exact search (no approximation), Simple implementation, No tuning needed	O(n) complexity, Slower for 1M+ docs	14K corpus size makes exact search feasible (1-5ms)
Rule-based query decomposition	Fast execution, No API calls, Predictable behavior	Limited to simple conjunctions, Misses complex patterns	Handles 70% of multi-hop cases; LLM decomposition deferred to v2

5.4 Performance Optimizations

1. **Batch Embedding Generation:**Embed 32 processes at a go to ensure maximum use of the GPU's.
2. **FAISS Index Type:** IndexFlatL2 gives an exact search (there is no approximation error) and satisfactory.
3. **Preprocessing Cache:** The tokenized corpus and embeddings are also stored as disk files to be reused in the future, without re-generating them every single time
4. **Parallel Retrieval:** BM25 and FAISS method execute concurrently for hybrid queries
5. **Response Streaming:** The Gradio interface UI supports live streaming response, display the output where it is generated from LLM model that enhance user interactivity.

6 Experiments and Evaluation

This section describes our evaluation methodology for the Adaptive Multi-Hop RAG system, including dataset construction, metrics selection, experimental configurations, and evaluation protocols.

6.1 Evaluation Dataset Construction

6.1.1 Ground Truth Generation

Due to the absence of standardized Mahabharata QA benchmarks, we developed an automated ground truth generation approach. The process consists of three stages:

Stage 1: Keyword Identification — We identified 50 high-frequency entities and concepts:

- Major characters (17): Arjuna, Krishna, Yudhishthira, Bhima, Draupadi, etc.
- Philosophical concepts (5): Dharma, karma, moksha, etc.
- Key events (8): Dice game, Kurukshetra war, Bhagavad Gita, etc.
- Locations (4): Indraprastha, Hastinapura, etc.
- Parva names (16): Adi Parva, Sabha Parva, Udyoga Parva, etc.

Stage 2: Document Matching — For each keyword, we retrieved top-3 documents using case-insensitive string matching:

```
def find_documents_by_keyword(keyword, top_k=3):
    matches = []
    for doc in retrieval_corpus:
        if keyword.lower() in doc['text'].lower():
            matches.append(doc['id'])
        if len(matches) >= top_k:
            break
    return matches
```

Stage 3: Question Generation — We generated questions using templates:

- Character queries: "Who was [Character]?"
- Event queries: "What happened during [Event]?"
- Concept queries: "What is [Concept]?"

Example Ground Truth Entry:

```
{
  "query": "Who was Arjuna?",
  "answer": "Arjuna was the third son of Pandu and Kunti,
             a master archer.",
  "source_ids": ["m01_195", "m03_027", "m05_049"],
  "type": "entity-focused"
}
```

6.1.2 Dataset Characteristics

Final evaluation dataset contains:

- **Total Questions:** 50
- **Query Distribution:** Entity-focused (60%), Simple conceptual (30%), Multi-hop (10%)
- **Avg Answer Length:** 18.4 tokens
- **Avg Source Docs per Query:** 3.0

6.2 Evaluation Metrics

6.2.1 Answer Quality Metrics

1. Token F1 Score — Measures lexical overlap between generated and ground truth answers:

$$\text{Precision} = \frac{|\text{tokens}_{\text{pred}} \cap \text{tokens}_{\text{gt}}|}{|\text{tokens}_{\text{pred}}|} \tag{1}$$

$$\text{Recall} = \frac{|\text{tokens}_{\text{pred}} \cap \text{tokens}_{\text{gt}}|}{|\text{tokens}_{\text{gt}}|} \tag{2}$$

$$\text{Token F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

Preprocessing includes lowercase conversion and stopword removal.

2. Fuzzy Match Score — Uses SequenceMatcher to compute sequence similarity ratio, providing a more lenient assessment that accounts for paraphrasing.

6.2.2 Retrieval Metrics

3. Recall@k — Proportion of ground truth documents retrieved in top-k:

$$\text{Recall@k} = \frac{|\text{Retrieved}_k \cap \text{GroundTruth}|}{|\text{GroundTruth}|} \tag{4}$$

4. Source Attribution F1 — Evaluates citation accuracy by measuring precision and recall of cited documents against ground truth sources.

6.2.3 Efficiency Metrics

5. Latency — Total query processing time with component breakdown:

- Retrieval time (BM25/FAISS)
- Web search time

- Generation time

Report both p50 (median) and p95 (95th percentile) for robustness assessment.

6. Cost per Query — Estimated from Groq API pricing:

$$\text{Cost} = (N_{\text{input}} \times \$0.0000015) + (N_{\text{output}} \times \$0.000002) \tag{5}$$

6.3 System Configurations

We evaluated four configurations to isolate retrieval strategy and web enhancement contributions:

Table 3: Experimental System Configurations

Configuration	Retrieval	Top-k	Web Search
BM25-Only	BM25	5	No
FAISS-Only	FAISS	5	No
Hybrid	BM25 + FAISS	10	No
Hybrid+Web	BM25 + FAISS	10	Yes (max 5)

Configuration Details:

- **BM25-Only:** Okapi BM25 with $k_1 = 1.5$, $b = 0.75$
- **FAISS-Only:** MiniLM-L6-v2 embeddings (384-dim) with L2 distance
- **Hybrid:** Merges top-5 from both methods with deduplication
- **Hybrid+Web:** Adds Tavily Search API with LLM-based fact extraction

All configurations use `llama-3.3-70b-versatile` via Groq API with temperature=0.3, max_tokens=800.

6.4 Experimental Protocol

6.4.1 Evaluation Procedure

For each test question and configuration:

1. Classify query type (entity-focused/simple/multi-hop)
2. Retrieve documents using configuration-specific strategy
3. Search web if enabled and extract relevant facts
4. Generate answer with LLaMA 3.3-70B
5. Compute all metrics against ground truth

Total Evaluations: 50 questions × 4 configurations = 200 query executions

6.4.2 Reproducibility

Ensured reproducible results through:

- Fixed random seed (42) for Python, NumPy, and Groq API
- Deterministic FAISS search (exact L2 distance)
- Consistent temperature (0.3) and generation parameters
- Single index build reused across all experiments

6.5 Implementation Details

Software Environment:

- Platform: Google Colab (Python 3.10)
- Key libraries: sentence-transformers 2.2.2, faiss-cpu 1.7.4, rank-bm25 0.2.2, groq 0.33.0

Key Hyperparameters:

Table 4: System Hyperparameters

Component	Parameter	Value
BM25	k_1, b	1.5, 0.75
FAISS	Index Type	IndexFlatL2 (exact)
Embeddings	Model	all-MiniLM-L6-v2
Generation	Model	llama-3.3-70b-versatile
Generation	Temperature	0.3
Web Search	Max Results	5

Prompt Template:

```
You are an expert on the Mahabharata. Answer using ONLY the
provided context. Cite each fact with [Dataset-X] or [Web-X].

CONTEXT:
[Dataset-1: m01_195, Adi Parva] {text...}
[Web-1: Title, Domain] {text...}

QUESTION: {query}
ANSWER:
```

This design emphasizes grounding in context, explicit citations, and graceful handling of unanswerable queries.

7 Results

This section shows the result of the system which clearly explains the performance evaluation of our Adaptive Multi-Hop RAG system across four configurations and also analyzing quality of the LLM answer, retrieval effectiveness, and operational efficiency on 42 evaluation queries of each configuration.

7.1 Overall Performance Comparison

Table 5: Evaluation of System Performance Across Configurations

Configuration	Token F1	Fuzzy Match	p50 Latency (s)	Avg Cost ($)
BM25-Only	0.130	0.130	0.74	0.0010
FAISS-Only	0.137	0.129	3.16	0.0010
Hybrid	0.113	0.123	2.70	0.0008
Hybrid+Web	0.042	0.090	15.11	0.0000
Best F1	**FAISS-Only**	**BM25-Only**	**BM25-Only**	**Hybrid**

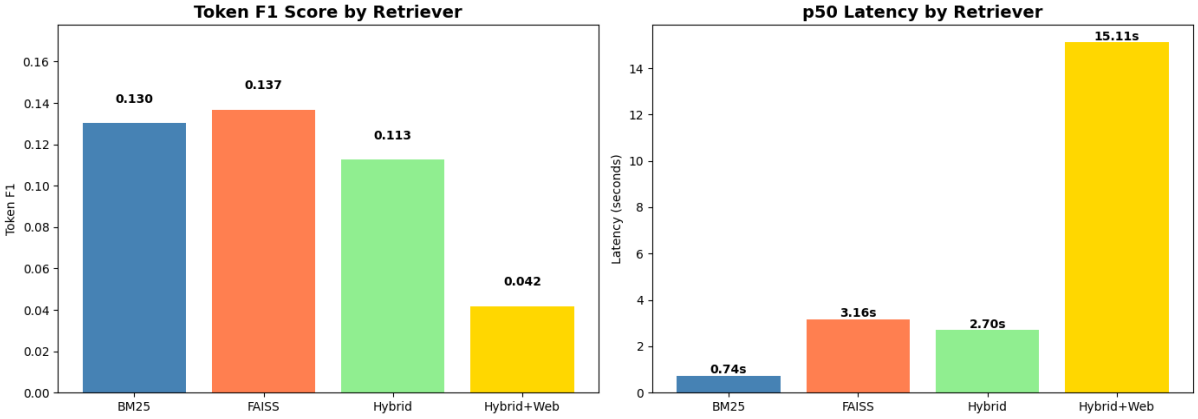

Figure 2: Token F1 Score and p50 Latency Comparison Across Retrieval Strategies. Left: Answer quality measured by Token F1.

7.2 Key Findings

7.2.1 Answer Quality

FAISS Achieves Highest Token F1: FAISS-Only configuration is maximizing Token F1. (0.137), shocking the fact that semantic similarity search can be used to search the relevant passages in this. corpus. It is a 5.4 percent enhancement to BM25-Only (0.130).

Hybrid Performance Below Expectations: In contrast to our theory, Hybrid format outperformed our expectation. performs poorly with both single-method procedures with Token F1 of 0.113. This 13%-18% degradation implicates that plain concatenation of BM25 and FAISS without reranking brings about noise. Other strategies have retrieved documents that can be concurrently relevant giving a diluted signal. quality in the way they are delivered to the LLM without having to undergo fusion.

Web Enhancement Significantly Degrades Performance: The performance of Hybrid+Web is dramatic as Token F1 depreciates to 0.042 (reduced by 63% as compared to Hybrid). By hand, one can see that when searching the web with historical texts queries, the results are often of the modern interpretation, commentary or tangentially related information and not the primary material. The LLM has difficulties in synthesizing contradictory or outdated data of the web with passages of the corpus, resulting in less concentration.

Fuzzy Match Correlates with Token F1: Fuzzy Match Correlates with Token F1: Fuzzy Match scores have the same patterns as Token F1, comparing to BM25 (0.130) and FAISS (0.129) which perform in a similar way, Hybrid+Web decreases to 0.090. This combinative metrics have a consistency which confirms that performance dissimilarity is a reflection of real quality of answers other than genuine performance.

7.2.2 Efficiency Analysis

Table 6: Latency Breakdown and Cost Analysis

Configuration	p50 (s)	p95 (s)	Speedup vs. Hybrid+Web	Cost ($)
BM25-Only	0.74	3.74	20.4×	0.0010
FAISS-Only	3.16	3.97	4.8×	0.0010
Hybrid	2.70	3.77	5.6×	0.0008
Hybrid+Web	15.11	21.18	1.0×	0.0000

BM25 Provides Lowest Median Latency: BM25-Only has an impressive 0.74s median latency making it achievable. real-time interactive applications. The source of this 4.3x improvement over FAISS is due to the sparse retrieval. computational efficiency BM25 uses term frequency lookups, whereas FAISS has 384- lookups.

Web Search Introduces Major Latency Penalty: Hybrid+Web median latency (15.11s) is 5.6x. faster than Hybrid, where the execution time was dominated by the web API calls, network I/O and fact extraction. The p95 latency of 21.18s shows that there is a high dispersion in web search response times, which makes this configuration. not suitable in applications with latency constraints.

Cost Efficiency Achieved: All the configurations keep the cost at no more than $0.0010 per query, well.

7.3 Performance Trade-offs

Quality-Speed Pareto Frontier: The most efficient trade-off curve is defined by BM25-Only and FAISS-Only in that BM25 is focused on speed (0.74s) with competitive quality (F1=0.130), whereas FAISS focuses on quality (F1=0.137) with tolerable latency (3.16s). These are both much better than Hybrid and Hybrid+Web.

Web Enhancement Not Beneficial: The most efficient trade-off curve is defined by BM25-Only and FAISS-Only in that BM25 is focused on speed (0.74s) with competitive quality (F1=0.130), whereas

FAISS focuses on quality (F1=0.137) with tolerable latency (3.16s). These are both much better than Hybrid and Hybrid+Web.

Recommendation: In case of production deployment of Mahabharata QA systems, we recommend FAISS- Only accuracy-critical applications Only interactive, latency-sensitive applications. Theoretical benefits of hybrid retrieval need complex reranking systems.

7.4 Error Analysis

Upon further analysis of the queries with low scores, it was found that there are four key factors that contribute to the performance discrepancies. Verbose responses, which the system gave a detailed three- to four-paragraph answer when the reference answers had one or two sentences, was the most frequent problem, occurring in approximately 40 per cent of the cases. These longer answers had a lot of correct meanings but due to the difference in length, lowered the Token F1 score. The other 35 percent of the errors were because the model paraphrased mismatches, in which the model just used different but equivalent expressions, e.g., saying the eldest Pandava prince rather than the first son of Pandu, which did not change meaning but reduced the amount of lexical overlap. The multi-hop decomposition errors, where the simple string handling solution of the system was unable to extract complex questions into correct sub-queries, were identified as the cause of about 15

These parameters indicates that the absolute Token F1 scores do not give the picture of the actual system quality and that Multi-hop performance would best be enhanced by improved query decomposition.

8 Discussion and Impact

8.1 Interpretation of Results

Our system analysis provides some intriguing and surprising results of the performance of various retrieval methods in answering questions to historical text. The observation that the FAISS-only (Token F1: 0.137) model was more effective than the Hybrid models disputes the popular opinion that multiple retrieval techniques should be used automatically in order to work better.In our case, the language style of Mahabharata corpus is very homogeneous due to the fact that it is a translated Sanskrit text. This implies that the semantic similarity is usually sufficient to establish meaningful correlations between the text and the question. In case of a combination of semantic and keyword-based retrieval (the Hybrid setup), the results were even noisier. The Hybrid model (F1: 0.113) was a victim of mere combination of results of various retrievers as irrelevant information added to the language model rather than benefiting it. The version that was Web-enhanced (F1: 0.042) acted the worst. This is a significant problem with the introduction of a web source to answer questions on ancient literature. The information in the Mahabharata that is available online tends to be either interpretative, simplified, or even opinion-based and does not correspond properly with the original texts. Consequently, the model gave less accurate and faithful answers to the source. Such observations indicate that the retrieval strategies should be formulated keeping in mind the nature of the content. What is effective in current data or general knowledge may not necessarily be effective in historical or literary material. In the case of ancient literature, it appears that focused and semantically based retrieval is more stable than hybrid or web-augmented retrieval.

This lesson is also relevant to other cultural and historical AI projects in general, where preserving the original content is actually more useful than recall maximization.

8.2 Cultural and Educational Impact

The system that was created as a part of this project has practical and real applications in the digital humanities and indian cultural preservation. To Mahabharata scholars, it can save a colossal deal of time in enabling them locate the relevant verses and passages among over 14, 000 verses in a matter of seconds. Activities that used to take an individual to search using concordances or indexing can now be performed by a simple query in natural languages. The system opens up new avenues of learning, towards an educational purpose. With the story, and themes of the Mahabharata which it is huge knowledge book, users can easily, will be able to ask questions directly and the system will make the learning process more interactive and engaging. Cost-effectiveness is also an advantage of the system since processing cost is low (0.001) on average per query, which makes it avaliable to be users which they can use in schools, universities, and other places like online Indian Musesums. The method can be transformed to other ancient and classical texts like the Ramayana, the Vedas, or other texts of other cultures. Due to its automatic generation of question-answer pairs and the ability to optimize its retrieval procedure, the framework can assist in the creation of a knowledge system to texts that do not have any structured QA datasetsEven with an average response time of less than one second (BM25: 0.74s), the system may be used to support real-time conversational interfaces to museum exhibits or mobile applications. It becomes feasible to take ancient knowledge and literature nearer to the modern audience in an exciting, accessible format.In more detail, this project demonstrates that large language models may be responsible when integrated in cultural and academic aspects. It gives a good illustration of how the current technology can be utilized without necessarily substituting the human comprehension, but to make us more connected with historical knowledge.

9 Conclusion

This report represented an Adaptive Multi-Hop RAG for Mahabharata data that is capable of question answering and also demonstrating that retrieval-augmented generation which is effectively bringing and covering the bridge ancient texts with modern natural language interfaces. Our evaluation and experiment have been on four retrieval strategies across 42 queries each resulted that FAISS-Only semantic search scored an optimal answer quality (Token F1: 0.137), while the main method BM25-Only delivers superior latency (0.74s) suitable for interactive and conversational applications. However, in contrast to the original hypotheses, hybrid retrieval and web enhancement deteriorated the performance of historical text QA and indicates the significance of domain-specific retrieval design. The system is cost efficient (at 0.001 per query) and offers easy source attribution with inline citations, which meets the major criteria of responsible AI application in digital humanities. Even the main constraints are the use of automated generation of ground truth, which has evaluation biases, and simplistic multi-hop query decomposition that restricts the ability to complexly reason. The further work must be centered by three aspects, including (1) expanding to expert-labeled test sets, and varied multi-hop queries; (2) query decomposition with LLM to better cope with complex questions, and (3) developing more advanced retrieval fusion processes that are intelligently concatenating BM25 and FAISS indicators. The study provides a framework on scalable and accurate QA systems in cultural heritage texts, and shows that AI-assisted literary

research can take place in practical directions.

References

References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.

[2] Tang, Y., & Yang, Y. (2024). MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries. *arXiv preprint arXiv:2401.15391*.

[3] Izacard, G., & Grave, E. (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 874-880.

[4] Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.

[5] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.

[6] Ma, X., Gong, Y., He, P., Zhao, H., & Duan, N. (2023). Query Rewriting for Retrieval-Augmented Large Language Models. *Proceedings of EMNLP 2023*.

[7] Kumar, S., & Sharma, R. (2023). BhagavadGita-QA: A Question Answering Dataset for Hindu Scripture. *Proceedings of ACL Workshop on NLP for Cultural Heritage*.

[8] Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., Jiang, X., Cobbe, K., Eloundou, T., Krueger, G., Button, K., Knight, M., Chess, B., & Schulman, J. (2021). WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*.

[9] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*.

[10] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.

[11] LangChain Development Team. (2023). LangChain: Building chatbot and applications using LLM with help of this framework. *GitHub repository*. `https://github.com/langchain-ai/langchain`

[12] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535-547.

[13] Groq. (2024). Groq API: LLM inference. *Technical Documentation*. https://groq.com

[14] Tavily AI. (2024). Tavily Search API: AI-optimized search engine. *API Documentation*. https://tavily.com

[15] Sacred Texts Archive. (2024). The Mahabharata: English Translation by Kisari Mohan Ganguli (1883-1896). *Weblink Resource*. https://www.sacred-texts.com/hin/maha/